Complete DSA Cheatsheet

26 Problems: Trees, Graphs, Backtracking, DP

Core Concepts & Fundamentals

References vs Values

Objects = References (shared)

```
int[] arr = new int[5];
helper(arr);  // Shared - modifications persist
arr[0] = 10;  // Changes everywhere!

List<Integer> list = new ArrayList<>();
backtrack(list);  // Must undo changes!
```

Primitives = Values (copied)

```
int x = 5;
helper(x);  // Copy passed, original unchanged
```

Why matters: This is why backtracking needs explicit undo!

Recursion Fundamentals

Three Components:
1. Base case (when to stop)
2. Recursive case (self-call)
3. Progress (move toward base)

Input vs Output:
Parameters = INPUT, Return = OUTPUT

```
if (n == 0) return 1;  // Correct!
// n=0 is INPUT, 1 is OUTPUT (ways/value)
// NOT "0 steps equals 1 step"!
```

Call Stack: Each call has own frame, pauses until child returns.

DFS vs BFS

DFS: Stack/Recursion, O(h) space, go deep
Use for: All paths, backtracking, tree problems
BFS: Queue, O(w) space, level by level
Use for: Shortest path, level order, min depth
Key: BFS finds shortest FIRST, DFS explores ALL!

Backtracking Pattern

Template:

```
void backtrack(state, choices) {
    if (complete) { save(state); return; }
    for (choice : choices) {
        make_choice();
        backtrack(state);
        undo_choice();  // Critical!
    }
}
```

When NO undo needed: Immutable strings
When undo needed: Lists, arrays, boolean[]

DP Fundamentals

When to use:
1. Optimal substructure
2. Overlapping subproblems

Top-Down: Problem → base (recursion + memo)
Bottom-Up: Base → problem (iteration + dp[])
Evolution: Recursion → Memo → Iterative → Space-opt
Space Optimization:
Possible when FIXED look-back (stairs: i-1, i-2)
Impossible when VARIABLE look-back (coins: i-coin)

DP: Bounded vs Unbounded

Unbounded (can reuse):
- Coin Change
- Forward iteration in 1D

Bounded (use once):
- 0/1 Knapsack
- BACKWARD iteration in 1D

Why backward? Prevents reusing same item by reading old values only!

Data Structures Reference

Map.Entry: Iterate over map entries

```
for (Map.Entry<K,V> entry : map.entrySet()) {
    K key = entry.getKey();
    V value = entry.getValue();
}
```

PriorityQueue: Min-heap by default

```
PriorityQueue<Integer> pq = new PriorityQueue<>()
    ;
pq.offer(5);  // Add
pq.poll();    // Remove min
pq.peek();    // View min

// Max-heap:
PriorityQueue<Integer> maxHeap =
    new PriorityQueue<>(Collections.reverseOrder
    ());
```

List of Lists:

```
List<List<Integer>> result = new ArrayList<>();
List<Integer> level = new ArrayList<>();
result.add(new ArrayList<>(level));  // Copy!
```

HashMap + List:

```
map.computeIfAbsent(k, x -> new ArrayList<>()).
    add(v);
```

Tree Patterns

Global Variable Pattern:
Track one thing, return another

```
class Solution {
    int diameter = 0;  // Track this
    int height(Node n) {
        diameter = max(diameter, left+right);
        return 1 + max(left, right);  // Return
    this
    }
}
```

Array Parameter Pattern:

```
int[] result = new int[1];
helper(root, result);
```

▷ 1. Climbing Stairs (70)

Ways to reach top (1 or 2 steps at a time)

Approach 1: Recursion + Memoization

Time: O(n) **Space:** O(n)

Each step calculated once. Memo array + recursion stack = O(n).

```java
public int climbStairs(int n) {
    int[] memo = new int[n + 1];
    return helper(n, memo);
}

int helper(int n, int[] memo) {
    if (n == 0 || n == 1) return 1;
    if (memo[n] != 0) return memo[n];

    memo[n] = helper(n-1, memo) + helper(n-2, memo);
    return memo[n];
}
```

Approach 2: Bottom-Up

Time: O(n) **Space:** O(n)

Single loop. DP array size n.

```java
public int climbStairs(int n) {
    if (n <= 1) return 1;
    int[] dp = new int[n + 1];
    dp[0] = 1; dp[1] = 1;

    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i-1] + dp[i-2];
    }
    return dp[n];
}
```

Approach 3: Space-Optimized

Time: O(n) **Space:** O(1)

Only need last 2 values! Fixed look-back.

```java
public int climbStairs(int n) {
    if (n <= 1) return 1;
    int prev2 = 1, prev1 = 1;

    for (int i = 2; i <= n; i++) {
        int cur = prev1 + prev2;
        prev2 = prev1;
        prev1 = cur;
    }
    return prev1;
}
```

Gotcha: Base case returns 1 (ways), not 0 (steps)!

Pattern: f(n) = f(n-1) + f(n-2) (Fibonacci)

▷ 2. House Robber (198)

Max money without robbing adjacent houses

Approach 1: Recursion + Memo

Time: O(n) **Space:** O(n)

Each house considered once. Memo + stack = O(n).

```java
public int rob(int[] nums) {
    int[] memo = new int[nums.length];
    Arrays.fill(memo, -1);
    return robFrom(0, nums, memo);
}

int robFrom(int i, int[] nums, int[] memo) {
    if (i >= nums.length) return 0;
    if (memo[i] != -1) return memo[i];

    int rob = nums[i] + robFrom(i+2, nums, memo);
    int skip = robFrom(i+1, nums, memo);

    memo[i] = Math.max(rob, skip);
    return memo[i];
}
```

Approach 2: Bottom-Up

Time: O(n) **Space:** O(n)

```java
public int rob(int[] nums) {
    if (nums.length == 1) return nums[0];
    int[] dp = new int[nums.length];
    dp[0] = nums[0];
    dp[1] = Math.max(nums[0], nums[1]);

    for (int i = 2; i < nums.length; i++) {
        dp[i] = Math.max(nums[i] + dp[i-2], dp[i-1])
    ;
    }
    return dp[nums.length - 1];
}
```

Approach 3: Space-Optimized

Time: O(n) **Space:** O(1)

Only need dp[i-1] and dp[i-2]!

```java
public int rob(int[] nums) {
    if (nums.length == 1) return nums[0];
    int prev2 = nums[0];
    int prev1 = Math.max(nums[0], nums[1]);

    for (int i = 2; i < nums.length; i++) {
        int cur = Math.max(nums[i] + prev2, prev1);
        prev2 = prev1;
        prev1 = cur;
    }
    return prev1;
}
```

Gotcha: NOT even/odd pattern! It's optimal choice at each house.

Choice: Rob this + skip next, OR skip this

▷ 3. Coin Change (322)

Minimum coins to make amount

Approach 1: Recursion + Memo

Time: O(amt × coins) **Space:** O(amt)

amt subproblems, each tries all coins. Memo + stack = O(amt).

```java
public int coinChange(int[] coins, int amount) {
    int[] memo = new int[amount + 1];
    Arrays.fill(memo, -1);
    int res = helper(coins, amount, memo);
    return res == Integer.MAX_VALUE ? -1 : res;
}

int helper(int[] coins, int amt, int[] memo) {
    if (amt == 0) return 0;
    if (amt < 0) return Integer.MAX_VALUE;
    if (memo[amt] != -1) return memo[amt];

    int min = Integer.MAX_VALUE;
    for (int coin : coins) {
        int res = helper(coins, amt - coin, memo);
        if (res != Integer.MAX_VALUE) {
            min = Math.min(min, 1 + res);
        }
    }
    memo[amt] = min;
    return min;
}
```

Approach 2: Bottom-Up

Time: O(amt × coins) **Space:** O(amt)

Fill amt entries, try each coin. No recursion overhead.

```java
public int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);  // "Infinity"
    dp[0] = 0;

    for (int i = 1; i <= amount; i++) {
        for (int coin : coins) {
            if (i >= coin) {
                dp[i] = Math.min(dp[i], 1 + dp[i -
    coin]);
            }
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}
```

Gotcha: Use amount+1 as infinity, NOT Integer.MAX_VALUE (overflow)!

Gotcha: The "1+" is the coin we JUST used in this step!

Unbounded: Can reuse coins! Forward iteration. Can't space-optimize (variable look-back).

▷ 4. 0/1 Knapsack

Max value, each item used 0 or 1 times

Approach 1: 2D Bottom-Up

Time: O(n × W) **Space:** O(n × W)

n items × W capacities. Each cell O(1) work.

```java
public int knapsack(int W, int[] val, int[] wt) {
    int n = wt.length;
    int[][] dp = new int[n + 1][W + 1];

    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            int skip = dp[i-1][w];
            int take = 0;
            if (wt[i-1] <= w) {
                take = val[i-1] + dp[i-1][w - wt[i
    -1]];
            }
            dp[i][w] = Math.max(skip, take);
        }
    }
    return dp[n][W];
}
```

Approach 2: 1D Optimized

Time: O(n × W) **Space:** O(W)

Only need previous row! But MUST iterate backward.

```java
public int knapsack(int W, int[] val, int[] wt) {
    int n = wt.length;
    int[] dp = new int[W + 1];

    for (int i = 0; i < n; i++) {
        // CRITICAL: BACKWARD iteration!
        for (int w = W; w >= wt[i]; w--) {
            dp[w] = Math.max(
                dp[w],
                val[i] + dp[w - wt[i]]
            );
        }
    }
    return dp[W];
}
```

Gotcha: Forward iteration = unbounded (wrong)! Backward = 0/1 (correct)!

Gotcha: Index mapping: dp[i] uses array[i-1] when dp is 1-indexed!

Bounded: Each item once! BACKWARD prevents reuse.

▷ 5. Level Order (102)

Binary tree nodes grouped by level

Approach: BFS with Queue

Time: O(n) **Space:** O(w)

Visit each node once. Queue holds max width (worst: n/2).

```java
public List<List<Integer>> levelOrder(TreeNode root)
    {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;

    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);

    while (!q.isEmpty()) {
        int size = q.size();  // MUST capture!
        List<Integer> level = new ArrayList<>();

        for (int i = 0; i < size; i++) {
            TreeNode node = q.poll();
            level.add(node.val);

            if (node.left != null) q.offer(node.left
    );
            if (node.right != null) q.offer(node.
    right);
        }
        result.add(level);
    }
    return result;
}
```

Gotcha: Must capture size BEFORE loop! q.size() changes!

Gotcha: New list EACH iteration, not reuse!

Pattern: Capture size, process level, add children

▷ 6. Diameter (543)

Longest path between any two nodes

Approach 1: Global Variable

Time: O(n) **Space:** O(h)

Visit each node once, O(1) work per node. Recursion stack = height.

```java
class Solution {
    int diameter = 0;

    public int diameterOfBinaryTree(TreeNode root) {
        height(root);
        return diameter;
    }

    private int height(TreeNode node) {
        if (node == null) return 0;

        int left = height(node.left);
        int right = height(node.right);

        diameter = Math.max(diameter, left + right);
        return 1 + Math.max(left, right);
    }
}
```

Approach 2: Array Parameter

```java
public int diameterOfBinaryTree(TreeNode root) {
    int[] diameter = new int[1];
    height(root, diameter);
    return diameter[0];
}

private int height(TreeNode node, int[] diameter) {
    if (node == null) return 0;

    int left = height(node.left, diameter);
    int right = height(node.right, diameter);

    diameter[0] = Math.max(diameter[0], left + right
    );
    return 1 + Math.max(left, right);
}
```

Gotcha: Must RETURN height, TRACK diameter separately!

Pattern: Return one thing, track another with global/array

▷ 7. Validate BST (98)

Check if valid binary search tree

Approach: Recursive with Bounds

Time: O(n) **Space:** O(h)

Visit each node once, O(1) check. Stack depth = height.

```java
public boolean isValidBST(TreeNode root) {
    return valid(root, Long.MIN_VALUE, Long.
    MAX_VALUE);
}

boolean valid(TreeNode node, long min, long max) {
    if (node == null) return true;

    if (!(min < node.val && node.val < max)) {
        return false;
    }

    return valid(node.left, min, node.val) &&
           valid(node.right, node.val, max);
}
```

Gotcha: Use Long.MIN/MAX! Integer bounds might be node values!

Gotcha: Strict inequality! Left < root < right.

Pattern: Pass bounds down, narrow at each level

▷ 8. Reverse Linked List (206)

Reverse singly linked list

Approach 1: Iterative

Time: O(n) **Space:** O(1)

Single pass. Only 3 pointers = O(1) space.

```java
public ListNode reverseList(ListNode head) {
    ListNode prev = null;
    ListNode curr = head;

    while (curr != null) {
        ListNode next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

Approach 2: Recursive
Time: O(n) **Space:** O(n)
Visit each once. Call stack = n frames.

```java
public ListNode reverseList(ListNode head) {
    if (head == null || head.next == null) return
    head;

    ListNode newHead = reverseList(head.next);

    head.next.next = head;
    head.next = null;

    return newHead;
}
```

Gotcha: newHead is SAME across all frames! head is different!
Gotcha: Pointer changes are PERMANENT! Nodes stay modified!
Iterative: 3 pointers. Recursive: fix on way back up.

▷ 9. Number of Islands (200)

Count connected 1's in grid
Approach: DFS
Time: O(m × n) **Space:** O(m × n)
Visit each cell once. DFS depth = entire grid worst case.

```java
public int numIslands(char[][] grid) {
    if (grid == null) return 0;
    int count = 0;

    for (int i = 0; i < grid.length; i++) {
        for (int j = 0; j < grid[0].length; j++) {
            if (grid[i][j] == '1') {
                dfs(grid, i, j);
                count++;
            }
        }
    }
    return count;
}

private void dfs(char[][] grid, int i, int j) {
    if (i < 0 || i >= grid.length || j < 0 ||
        j >= grid[0].length || grid[i][j] == '0') {
        return;
    }

    grid[i][j] = '0';  // Mark visited

    dfs(grid, i + 1, j);
    dfs(grid, i - 1, j);
    dfs(grid, i, j + 1);
    dfs(grid, i, j - 1);
}
```

Gotcha: Modifies input! Each DFS marks entire island as '0'.
Pattern: Each DFS = one island. Count DFS calls.

▷ 10. Letter Combinations (17)

Phone number letter combinations
Approach: Backtracking
Time: O(4^n) **Space:** O(n)
Each digit ≤ 4 letters. All combos = 4^n. Depth = n.

```java
String[] map = {"", "", "abc", "def", "ghi",
    "jkl", "mno", "pqrs", "tuv", "wxyz"};

public List<String> letterCombinations(String digits
    ) {
    List<String> res = new ArrayList<>();
    if (digits.isEmpty()) return res;
    backtrack(0, "", digits, res);
    return res;
}

private void backtrack(int i, String cur,
    String digits, List<String> res) {
    if (cur.length() == digits.length()) {
        res.add(cur);
        return;
    }

    String chars = map[digits.charAt(i) - '0'];
    for (char c : chars.toCharArray()) {
        backtrack(i + 1, cur + c, digits, res);
    }
}
```

Gotcha: No explicit undo! Strings are immutable!
Pattern: Try each choice, recurse, no undo needed

▷ 11. Permutations (46)

All arrangements of array
Approach: Backtracking with used[]
Time: O(n!) **Space:** O(n)
n! permutations, each takes O(n) to build. Stack = n.

```java
public List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, new ArrayList<>(),
        new boolean[nums.length], result);
    return result;
}

void backtrack(int[] nums, List<Integer> cur,
    boolean[] used, List<List<Integer>> result) {
    if (cur.size() == nums.length) {
        result.add(new ArrayList<>(cur));
        return;
    }

    for (int i = 0; i < nums.length; i++) {
        if (used[i]) continue;

        cur.add(nums[i]);
        used[i] = true;
        backtrack(nums, cur, used, result);
        cur.remove(cur.size() - 1);
        used[i] = false;
    }
}
```

Gotcha: Must COPY list when saving: new ArrayList<>(cur)!
Gotcha: Must undo BOTH: remove from list AND set used[i]=false!
Pattern: used[] prevents reuse. Explicit undo needed.

▷ 12. Subsets (78)

All possible subsets
Approach: Backtracking
Time: O(2^n) **Space:** O(n)
2^n subsets (include/exclude). Each O(n) to copy. Depth = n.

```java
public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(), result);
    return result;
}

void backtrack(int[] nums, int start,
    List<Integer> cur, List<List<Integer>> result) {
    result.add(new ArrayList<>(cur));  // Save at
    EVERY node!

    for (int i = start; i < nums.length; i++) {
        cur.add(nums[i]);
        backtrack(nums, i + 1, cur, result);
        cur.remove(cur.size() - 1);
    }
}
```

Gotcha: Save at EVERY node, not just leaves!
Gotcha: Use start index to avoid duplicates!
Pattern: Include/exclude each element. Save everywhere.

Templates

Backtracking

```java
void backtrack(state, choices) {
    if (complete) {
        save(state);  // Copy if needed!
        return;
    }

    for (choice : choices) {
        make_choice();
        backtrack(new_state);
        undo_choice();  // Critical for lists/
    arrays!
    }
}
```

BFS Level Order

```
Queue<Node> q = new LinkedList<>();
q.offer(root);

while (!q.isEmpty()) {
    int size = q.size();  // MUST capture!
    List<Integer> level = new ArrayList<>();  // New each time!

    for (int i = 0; i < size; i++) {
        Node node = q.poll();
        // Process node

        for (Node child : node.children) {
            q.offer(child);
        }
    }
    result.add(level);
}
```

DFS with Global

```
class Solution {
    int result = 0;  // Track this

    public int solve(TreeNode root) {
        helper(root);
        return result;
    }

    int helper(TreeNode node) {
        if (node == null) return base;

        int left = helper(node.left);
        int right = helper(node.right);

        result = update(result, left, right);
        return compute(left, right);  // Return this
    }
}
```

DP Memoization

```
public int solve(params) {
    int[] memo = new int[n];  // Create ONCE!
    Arrays.fill(memo, -1);
    return helper(params, memo);  // Pass to ALL calls
}

int helper(params, int[] memo) {
    if (base_case) return base_value;
    if (memo[i] != -1) return memo[i];  // Check cache

    memo[i] = compute using helper(smaller, memo);
    return memo[i];
}
```

DP Bottom-Up

```
public int solve(params) {
    int[] dp = new int[n];
    dp[0] = base0;
    dp[1] = base1;

    for (int i = 2; i < n; i++) {
        dp[i] = compute(dp[i-1], dp[i-2], ...);
    }

    return dp[n-1];
}
```

Common Mistakes - Expanded

Critical Errors to Avoid

1. Queue Size in Loop
× `for (int i = 0; i < q.size(); i++)`
✓ `int size = q.size(); for (int i = 0; i < size; i++)`
Why: q.size() changes as you add children!

2. Reusing List
× Create list outside while loop
✓ `List<Integer> level = new ArrayList<>();` inside each iteration
Why: All references point to same list!

3. Not Copying List
× `result.add(current);`
✓ `result.add(new ArrayList<>(current));`
Why: current keeps changing, need snapshot!

4. Missing Backtrack
× Add choice, recurse, no undo
✓ Add choice, recurse, undo choice
Why: State persists across branches!

5. Memo Inside Recursion
× `int[] memo = new int[n];` inside helper
✓ Create in main, pass to ALL calls
Why: Each call creates NEW memo!

6. Forward vs Backward Iteration
× `for (w = 0; w <= W; w++)` for 0/1 Knapsack
✓ `for (w = W; w >= wt[i]; w-)` for 0/1
Why: Forward allows reuse (unbounded)!

7. Input vs Output Confusion
× `if (n==0) return 0` for stairs
✓ `if (n==0) return 1`
Why: 0 is INPUT, 1 is OUTPUT (ways)!

8. Index Mapping
× `dp[i]` uses `array[i]` when dp is 1-indexed
✓ `dp[i]` uses `array[i-1]`
Why: DP row i = first i items = array[0..i-1]!

9. Using -1 for Impossible
× Return -1 during recursion in Coin Change
✓ Use MAX_VALUE, convert to -1 at end
Why: Math.min(-1, x) gives wrong result!

10. Integer vs Long Bounds
× `int min, int max` for BST validation
✓ `long min, long max`
Why: Node values might BE Integer.MIN/MAX!

11. Modifying Input
× Not realizing grid[i][j] = '0' modifies input
✓ Use visited[][] if input must stay intact
Why: Modifies original data structure!

12. Global Variable Scope
× Declaring result inside helper
✓ Instance variable or array parameter
Why: Must persist across all calls!

Complexity Quick Reference

Problem	Time	Space
Climb Stairs	O(n)	O(1)
House Robber	O(n)	O(1)
Coin Change	O(amt×c)	O(amt)
Knapsack	O(n×W)	O(W)
Level Order	O(n)	O(w)
Diameter	O(n)	O(h)
Validate BST	O(n)	O(h)
Reverse List	O(n)	O(1) / O(n)
Islands	O(mn)	O(mn)
Letter Comb	$O(4^n)$	O(n)
Permutations	O(n!)	O(n)
Subsets	$O(2^n)$	O(n)

Key Patterns Summary

Pattern Recognition

Counting (SUM): Climbing Stairs
Optimization (MAX/MIN): Robber, Coins, Knapsack
All Combinations: Letter Comb, Permutations, Subsets
Shortest Path: BFS (Level Order, Min Depth)
All Paths: DFS (Diameter, Islands)
Validation: BST with bounds
Track + Return Different: Diameter (global variable)
Unbounded: Coin Change (forward)
Bounded: Knapsack (backward)
Fixed Look-back: Space optimize to O(1)
Variable Look-back: Can't space optimize

Capture size! Copy lists! Backward for 0/1! Memo once! Long bounds! Input ≠ Output!